

Universidade Estadual da Paraíba – UEPB

Disciplina: Linguagem de Programação II

## Criando Objetos em Java: Classes, Atributos, Construtores e Métodos

### Parte 1: O que é uma classe?

#### A ideia central

Quando escrevemos um programa que precisa representar algo do mundo real — um aluno, um produto, um carro — precisamos de uma forma de organizar os dados desse “algo” dentro do código. É exatamente para isso que existem as **classes**.

#### Definição: Classe

Uma **classe** é um **molde** que descreve como um tipo de objeto vai ser. Ela define quais **dados** o objeto vai guardar e quais **ações** ele pode realizar. A classe em si não é o objeto: ela é a **receita** para criar objetos.

#### Analogia: a forma e o bolo

Pense numa forma de bolo. A forma não é um bolo — é o **molde** que define o formato de todos os bolos que serão feitos com ela. Cada bolo produzido a partir da mesma forma é um **objeto** diferente, mas todos têm a mesma forma básica. Em Java, a **classe** é a forma. Os **objetos** são os bolos. Você pode criar quantos objetos quiser a partir de uma única classe.

## A mesma classe cria objetos diferentes

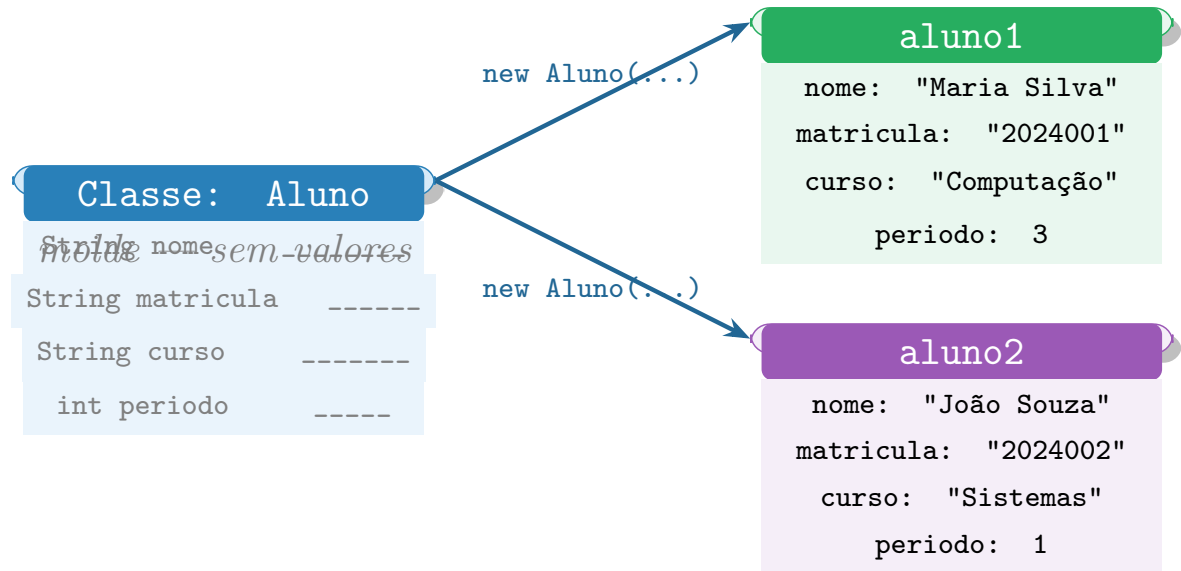


Figure 1: A classe `Aluno` é o molde. Cada objeto criado com `new` tem os mesmos *tipos* de dados, mas com *valores* diferentes e independentes.

### Como identificar uma classe num problema?

Leia o enunciado e procure os **substantivos principais** — as “coisas” que o sistema precisa conhecer. Cada uma dessas coisas é um candidato a classe.

*Exemplo:* “O sistema precisa cadastrar alunos, professores e turmas.” → Candidatos: Aluno, Professor, Turma.

## Parte 2: O que são atributos?

### Definição: Atributo

**Atributos** são as **informações** (dados) que cada objeto guarda sobre si mesmo. Eles ficam declarados dentro da classe, mas cada objeto tem os seus **próprios valores** para esses atributos.

### Analogia: a ficha de matrícula

Pense numa ficha de matrícula da universidade. A ficha tem *campos*: nome, CPF, curso, período. Esses campos são os **atributos** da classe. Quando um aluno preenche a ficha com os seus dados, temos um **objeto** — uma ficha específica de uma pessoa real.

## Como identificar atributos em um problema

Depois de identificar a classe, pergunte: “O que este objeto precisa saber sobre si mesmo?” As respostas são os atributos.

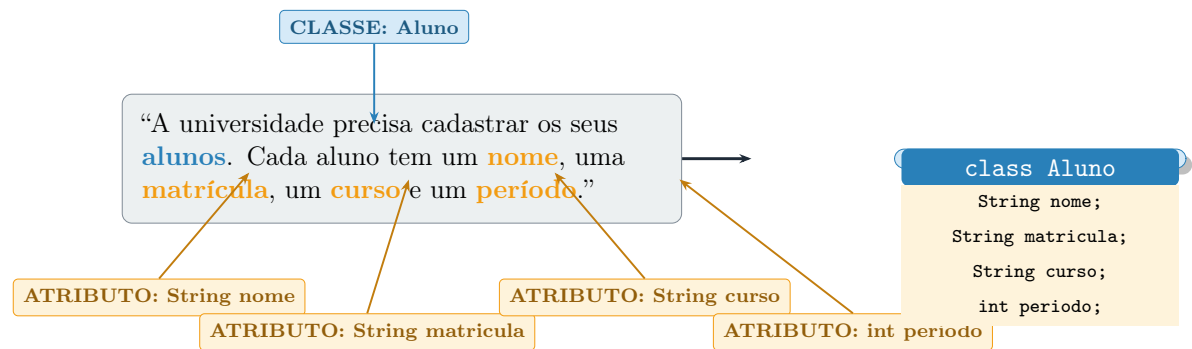


Figure 2: A partir do enunciado do problema, identifica-se a **classe** (substantivo principal) e os **atributos** (características que cada objeto precisa guardar).

## Tipos de dados comuns em atributos

| Tipo    | Quando usar                  | Exemplos de valores    |
|---------|------------------------------|------------------------|
| String  | Texto de qualquer tamanho    | "Maria", "LP2", "UEPB" |
| int     | Número inteiro (sem vírgula) | 3, 2024, -1, 0         |
| double  | Número com casas decimais    | 9.5, 3.14, -0.5        |
| boolean | Verdadeiro ou falso          | true, false            |
| char    | Um único caractere           | 'A', '?', '5'          |

### Como escolher o tipo certo?

- Nome, endereço, descrição, código com letras → **String**
- Período, quantidade, ano, idade → **int**
- Nota, preço, salário, peso → **double**
- Está aprovado? Está ativo? → **boolean**

```

1 public class Aluno {
2
3     // Atributos: os dados que todo objeto Aluno vai guardar
4     String nome;
5     String matricula;
6     String curso;
7     int periodo;
8
9 }

```

Listing 1: Classe Aluno com atributos declarados

### Verifique seu entendimento

Olhe para a classe `Aluno` acima. Ela está **declarada**, mas ainda não tem nenhum objeto criado. Lembre: a classe é o molde. Como criamos um objeto a partir dela? Você vai descobrir na Parte 4!

## Parte 3: O construtor

### O problema sem construtor

Imagine que você quer criar um aluno e preencher os seus dados. Sem um construtor, você teria que atribuir cada atributo manualmente:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         Aluno a = new Aluno(); // cria o objeto (ainda vazio)  
4         a.nome      = "Maria Silva";  
5         a.matricula = "2024001";  
6         a.curso     = "Computação";  
7         a.periodo   = 3;  
8     }  
9 }
```

Listing 2: Criando e preenchendo um aluno SEM construtor

Isso funciona, mas é **trabalhoso** e **propenso a erro**: se alguém esquecer de preencher um campo, o objeto ficará com dados incompletos. O **construtor** resolve isso.

### Definição: Construtor

Um **construtor** é um método especial que é chamado automaticamente no momento em que o objeto é criado com `new`. Ele permite **inicializar todos os atributos de uma vez**, de forma organizada, garantindo que o objeto já nasce com os dados corretos.

### Analogia: a ficha já pré-preenchida

Um construtor é como entregar a ficha de matrícula **já preenchida** no momento da inscrição, em vez de entregá-la em branco e preencher campo por campo depois. O objeto já nasce com tudo no lugar certo.

### Como escrever um construtor

```
1 public class Aluno {  
2  
3     String nome;  
4     String matricula;  
5     String curso;  
6     int    periodo;  
7  
8     // O construtor tem o mesmo nome da classe e nao tem tipo de  
9     // retorno
```

```

9     public Aluno(String nome, String matricula, String curso, int
10         periodo) {
11         this.nome      = nome;
12         this.matricula = matricula;
13         this.curso     = curso;
14         this.periodo   = periodo;
15     }

```

Listing 3: Classe Aluno com construtor

## A palavra this

### Definição: this

**this** é uma referência ao **próprio objeto** que está sendo criado. Usamos **this.nome** para dizer “o atributo **nome** deste objeto”, distinguindo-o do parâmetro **nome** que chegou pelo construtor.

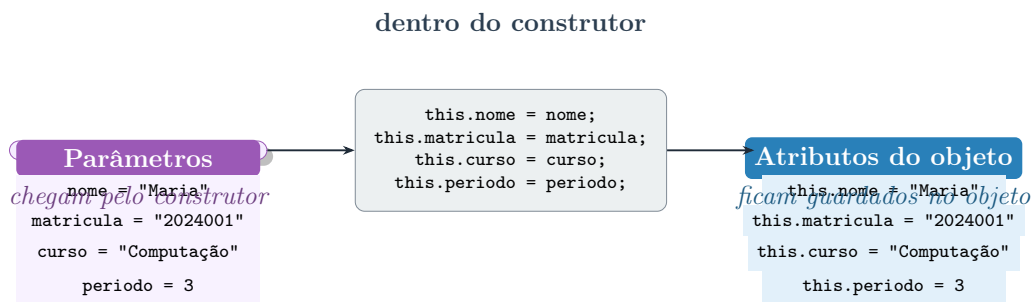


Figure 3: **this.nome** refere-se ao atributo do objeto; **nome** (sem **this**) é o parâmetro recebido pelo construtor. A instrução **this.nome = nome** copia o valor do parâmetro para o atributo.

### Quando você precisa de this?

**this** é necessário quando o **parâmetro do construtor tem o mesmo nome que o atributo**. Se você chamar o parâmetro de **nomeAluno** e o atributo de **nome**, não precisaria de **this**. Mas o padrão mais usado em Java é manter o mesmo nome (menos para memorizar!) e usar **this** para distinguir.

```

1 // COM this (nomes iguais -- mais comum)
2 public Aluno(String nome, int periodo) {
3     this.nome      = nome;    // this.nome = atributo; nome =
4     // parametro
5     this.periodo   = periodo;
6 }
7
8 // SEM this (nomes diferentes -- funciona, mas é menos
9 // convencional)
10 public Aluno(String nomeAluno, int periodoAluno) {
11     nome           = nomeAluno;

```

```
10     periodo = periodoAluno;
11 }
```

Ambas as versões funcionam. A primeira (com `this`) é a mais utilizada na prática.

## Parte 4: Criando objetos com new

### A palavra-chave new

`new` é a instrução que **cria um novo objeto** a partir de uma classe. Ela aloca espaço na memória para o objeto e chama automaticamente o construtor para preencher os atributos.

```
1 public class Main {
2     public static void main(String[] args) {
3
4         // Criando dois objetos da classe Aluno
5         Aluno a1 = new Aluno("Maria Silva", "2024001", "Computação", 3);
6         Aluno a2 = new Aluno("João Souza", "2024002", "Sistemas", 1);
7         Aluno a3 = new Aluno("Ana Ferreira", "2024003", "Computação", 5);
8
9         // Acessando atributos diretamente
10        System.out.println(a1.nome);           // Maria Silva
11        System.out.println(a2.periodo);        // 1
12        System.out.println(a3.matricula);      // 2024003
13    }
14 }
```

Listing 4: Criando objetos com construtor

### Lendo a linha de criação de objeto

```
Aluno a1 = new Aluno("Maria", "2024001", "Computação", 3);
```

- `Aluno`: o tipo do objeto (o nome da classe).
- `a1`: a variável que vai guardar a referência ao objeto.
- `new`: instrução que cria o objeto na memória.
- `"Maria", ...`: os valores passados para o construtor.

## Variáveis (no Main)

## Objetos na memória

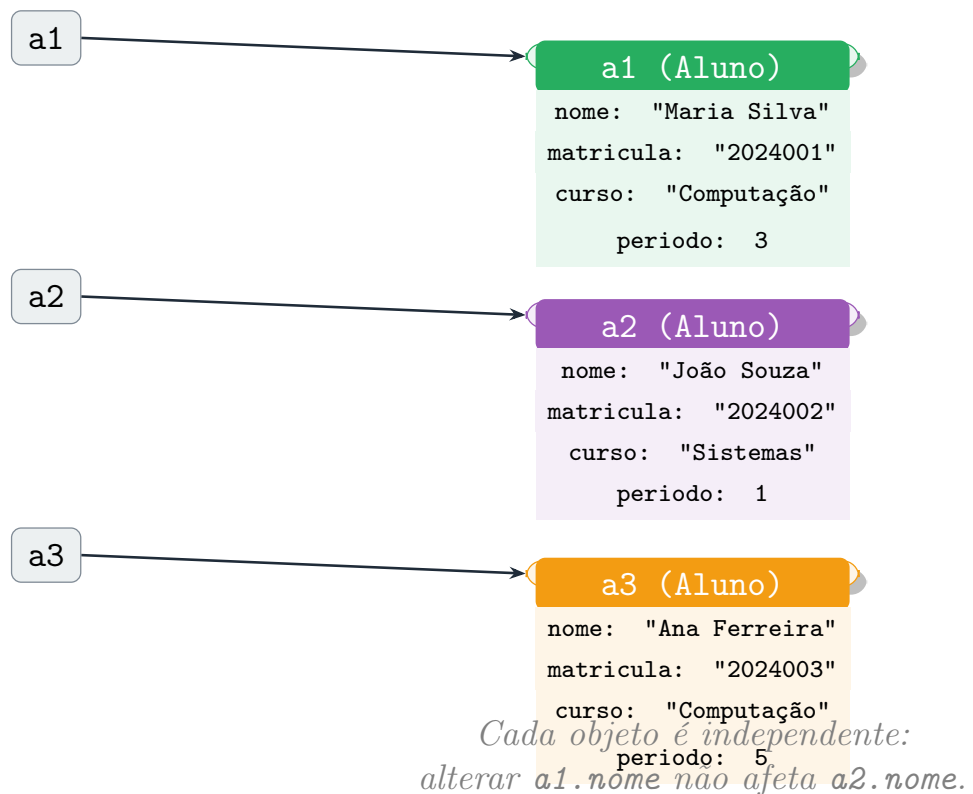


Figure 4: Cada variável (a1, a2, a3) aponta para um objeto diferente na memória. Os objetos são **independentes**: mudar um não altera os outros.

### Verifique seu entendimento

Se você executar `a1.periodo = 4;` após criar os objetos, o valor de `a2.periodo` muda também? Por quê?

## Parte 5: Métodos

### Definição: Método

Um **método** é uma **ação** que um objeto sabe realizar. Métodos ficam declarados dentro da classe e podem usar os atributos do próprio objeto. Para chamar um método, escrevemos: `objeto.nomeDoMetodo();`

### Método que exibe dados

O método mais comum para iniciantes é o que **exibe os dados** do objeto na tela. Por convenção, chamamos de `mostrarDados()`.

```
1 public class Aluno {  
2  
3     String nome;  
4     String matricula;
```

```

5      String curso;
6      int    periodo;
7
8      public Aluno(String nome, String matricula, String curso, int
9      periodo) {
10          this.nome      = nome;
11          this.matricula = matricula;
12          this.curso     = curso;
13          this.periodo   = periodo;
14      }
15
16      // Método: usa os atributos do proprio objeto para imprimir
17      void mostrarDados() {
18          System.out.println("Nome:      " + nome);
19          System.out.println("Matrícula: " + matricula);
20          System.out.println("Curso:     " + curso);
21          System.out.println("Período:   " + periodo);
22      }

```

Listing 5: Método mostrarDados() na classe Aluno

```

1 public class Main {
2     public static void main(String[] args) {
3         Aluno a1 = new Aluno("Maria Silva", "2024001", "Computação"
4         , 3);
5         Aluno a2 = new Aluno("João Souza", "2024002", "Sistemas",
6         1);
7
8         a1.mostrarDados(); // imprime os dados de Maria
9         System.out.println("---");
10        a2.mostrarDados(); // imprime os dados de João

```

Listing 6: Chamando o método no Main

#### Saída esperada

```

Nome:      Maria Silva
Matrícula: 2024001
Curso:     Computação
Período:   3
---
Nome:      João Souza
Matrícula: 2024002
Curso:     Sistemas
Período:   1

```

## Método que calcula e retorna um valor

Além de exibir dados, um método pode **calcular** e **devolver** um resultado usando a palavra `return`.

```
1 public class Produto {
2
3     String nome;
4     double preco;
5     int quantidade;
6
7     public Produto(String nome, double preco, int quantidade) {
8         this.nome = nome;
9         this.preco = preco;
10        this.quantidade = quantidade;
11    }
12
13    // Calcula e retorna o valor total do produto no estoque
14    double calcularTotal() {
15        return preco * quantidade;
16    }
17
18    void mostrarDados() {
19        System.out.println("Produto: " + nome);
20        System.out.println("Preço: R$ " + preco);
21        System.out.println("Qtd: " + quantidade);
22        System.out.printf("Total: R$ %.2f\n", calcularTotal());
23    }
24 }
```

Listing 7: Classe Produto com método de cálculo

```
1 public class Main {
2     public static void main(String[] args) {
3         Produto p1 = new Produto("Caderno", 12.50, 30);
4         Produto p2 = new Produto("Caneta", 2.00, 150);
5
6         p1.mostrarDados();
7         System.out.println("---");
8         p2.mostrarDados();
9
10        // Usando o retorno do método diretamente
11        double totalGeral = p1.calcularTotal() + p2.calcularTotal();
12
13        System.out.printf("Total geral: R$ %.2f\n", totalGeral);
14    }
15 }
```

Listing 8: Usando a classe Produto no Main

### Saída esperada

```
Produto: Caderno
Preço:   R$ 12.5
Qtd:     30
Total:   R$ 375,00
---
Produto: Caneta
Preço:   R$ 2.0
Qtd:     150
Total:   R$ 300,00
Total geral: R$ 675,00
```

### Método com void vs método com tipo de retorno

- **void**: o método faz algo (geralmente imprime) mas **não devolve** nenhum valor. Exemplo: `void mostrarDados()`.
- **com tipo**: o método **calcula e devolve** um valor. O tipo antes do nome diz qual tipo será devolvido. Exemplo: `double calcularTotal()` devolve um `double`.

## Método que modifica um atributo

Métodos também podem **alterar** os atributos de um objeto.

```
1 public class Carro {
2
3     String marca;
4     String modelo;
5     int ano;
6     double velocidade;
7
8     public Carro(String marca, String modelo, int ano) {
9         this.marca = marca;
10        this.modelo = modelo;
11        this.ano = ano;
12        this.velocidade = 0.0; // começa parado
13    }
14
15    void acelerar(double incremento) {
16        velocidade = velocidade + incremento;
17        System.out.println(modelo + " acelerou para " + velocidade
18        + " km/h");
19    }
20
21    void frear(double decremento) {
22        if (velocidade - decremento >= 0) {
23            velocidade = velocidade - decremento;
24        } else {
25            velocidade = 0;
26        }
27    }
28 }
```

```

25     }
26     System.out.println(modelo + " desacelerou para " +
velocidade + " km/h");
27 }
28
29 void mostrarDados() {
30     System.out.println(marca + " " + modelo + " (" + ano + ")")
;
31     System.out.println("Velocidade atual: " + velocidade + " km
/h");
32 }
33 }

```

Listing 9: Classe Carro com método que modifica velocidade

```

1 public class Main {
2     public static void main(String[] args) {
3         Carro c = new Carro("Fiat", "Uno", 2020);
4         c.mostrarDados();
5         c.acelerar(40);
6         c.acelerar(30);
7         c.frear(20);
8         c.mostrarDados();
9     }
10 }

```

Listing 10: Usando a classe Carro

#### Saída esperada

```

Fiat Uno (2020)
Velocidade atual: 0.0 km/h
Uno acelerou para 40.0 km/h
Uno acelerou para 70.0 km/h
Uno desacelerou para 50.0 km/h
Fiat Uno (2020)
Velocidade atual: 50.0 km/h

```

## Parte 6: Exemplo completo

Vamos juntar tudo num exemplo completo: uma classe `Aluno` com notas e um método para calcular a média.

```

1 public class Aluno {
2
3     String nome;
4     String matricula;
5     double nota1;
6     double nota2;
7
8     public Aluno(String nome, String matricula, double nota1,
double nota2) {

```

```

9      this.nome      = nome;
10     this.matricula = matricula;
11     this.nota1     = nota1;
12     this.nota2     = nota2;
13 }
14
15 double calcularMedia() {
16     return (nota1 + nota2) / 2;
17 }
18
19 void mostrarSituacao() {
20     double media = calcularMedia();
21     String situacao;
22     if (media >= 7.0) {
23         situacao = "Aprovado";
24     } else if (media >= 5.0) {
25         situacao = "Recuperação";
26     } else {
27         situacao = "Reprovado";
28     }
29     System.out.printf("%s (mat. %s) | Média: %.1f | %s\n",
30         nome, matricula, media, situacao);
31 }
32 }

```

Listing 11: Aluno.java — classe completa com cálculo de média

```

1 public class Main {
2     public static void main(String[] args) {
3
4         Aluno a1 = new Aluno("Maria Silva", "2024001", 8.0, 9.0);
5         Aluno a2 = new Aluno("João Souza", "2024002", 5.0, 6.0);
6         Aluno a3 = new Aluno("Ana Ferreira", "2024003", 3.0, 4.0);
7         Aluno a4 = new Aluno("Pedro Lima", "2024004", 7.0, 8.5);
8
9         System.out.println("==== Resultado da Turma =====");
10        a1.mostrarSituacao();
11        a2.mostrarSituacao();
12        a3.mostrarSituacao();
13        a4.mostrarSituacao();
14    }
15 }

```

Listing 12: Main.java — criando e exibindo vários alunos

#### Saída esperada

```

===== Resultado da Turma =====
Maria Silva (mat. 2024001) | Média: 8.5 | Aprovado
João Souza (mat. 2024002) | Média: 5.5 | Recuperação
Ana Ferreira (mat. 2024003) | Média: 3.5 | Reprovado
Pedro Lima (mat. 2024004) | Média: 7.8 | Aprovado

```

## Parte 7: Erros comuns

### Erro 1: esquecer o new

```
1 Aluno a = Aluno("Maria", "001", "CC", 3); // ERRO de compilação!
```

O `new` é obrigatório para criar um objeto. Sem ele, o Java não sabe que você quer criar um novo objeto — e não compila. **Correto:** `Aluno a = new Aluno("Maria", "001", "CC", 3);`

### Erro 2: usar o nome da classe como se fosse um objeto

```
1 Aluno.mostrarDados(); // ERRO: quem vai mostrar? Qual aluno?
```

Métodos são chamados em **objetos**, não na classe. A classe é o molde — você precisa criar um objeto antes de chamar métodos nele. **Correto:** `Aluno a = new Aluno(...); a.mostrarDados();`

### Erro 3: variável criada mas objeto não inicializado

```
1 Aluno a;  
2 a.mostrarDados(); // ERRO em tempo de execução:  
   NullPointerException
```

Declarar a variável (`Aluno a;`) **não cria** um objeto. A variável existe, mas não aponta para nenhum objeto ainda (vale `null`). Qualquer chamada de método em `null` trava o programa. **Correto:** `Aluno a = new Aluno(...); a.mostrarDados();`

### Erro 4: parâmetros fora de ordem no construtor

```
1 // Construtor: Aluno(String nome, String matricula, double nota1  
   , double nota2)  
2 Aluno a = new Aluno("2024001", "Maria", 8.0, 9.0); // ERRADO!  
3 // nome receberá "2024001" e matricula receberá "Maria"
```

O Java passa os valores **na ordem em que estão**, sem verificar o significado. Sempre confira a ordem dos parâmetros do construtor. **Correto:** `new Aluno("Maria", "2024001", 8.0, 9.0);`

## Exercícios

**Questão 1. Identificando classe e atributos.** Leia o enunciado abaixo e responda:

“Uma livraria quer cadastrar seus livros. Cada livro tem um título, um autor, um ano de publicação e um preço.”

- a) Qual é o nome da classe que você criaria?
- b) Quais seriam os atributos? Qual tipo de dado (`String`, `int`, `double`) você usaria para cada um?

- c) Escreva a declaração da classe em Java **apenas com os atributos** (sem construtor ainda).

**Questão 2. Escrevendo o construtor.** Escreva um construtor para a classe Livro da questão anterior. O construtor deve receber os quatro atributos como parâmetros e usar `this` para inicializá-los. Adicione também um método `mostrarDados()` que imprime título, autor, ano e preço.

**Questão 3. Criando objetos.** Usando a classe Livro completa, escreva um Main que crie **três livros** com dados fictícios, chame `mostrarDados()` em cada um e, por fim, imprima diretamente o título do segundo livro acessando o atributo `titulo`.

**Questão 4. Adicionando um método com retorno.** Adicione um método `double calcularDesconto(int porcentagem)` à classe Produto vista na Parte 5. O método deve calcular e retornar o preço com desconto. Fórmula: `preço - (preço * porcentagem / 100)`. Teste no Main imprimindo o preço original e o preço com 20% de desconto.

**Questão 5. Corrija os erros.** O código abaixo tem **três erros**. Identifique cada um e escreva a versão corrigida.

```
1 public class Main {
2     public static void main(String[] args) {
3         Carro c;
4         c.acelerar(50); //
5         erro 1
6         Carro d = Carro("Honda", "Civic", 2022); //
7         erro 2
8         Carro.mostrarDados(); //
9         erro 3
10    }
```

**Questão 6. Desafio: classe completa do zero.** Escreva, do início ao fim, uma classe `Funcionario` para um sistema de uma empresa. Cada funcionário tem nome (`String`), cargo (`String`) e salário mensal (`double`).

- Declare a classe com os atributos.
- Escreva o construtor recebendo os três atributos.
- Escreva o método `void mostrarDados()`.
- Escreva o método `double calcularSalarioAnual()` que retorna o `salário × 12`.
- No Main, crie dois funcionários, mostre seus dados e compare quem tem o maior salário anual usando um `if`.

### Checklist antes de entregar

- O nome da classe começa com letra **maiúscula**? (Aluno, não aluno)
- Os atributos estão declarados **dentro da classe** e **fora** dos métodos?
- O construtor tem o **mesmo nome** da classe?
- Você usou `this.atributo = parametro;` para cada atributo?
- Criou os objetos com `new` no Main?
- Os métodos void **não têm** `return` com valor?
- Os métodos com tipo de retorno têm um `return` com um valor do tipo certo?